Modern AI IDEs

Exploring the AI-powered development tool ecosystem

Presented by:
BEAUVAIS Luc
BRUNEAU Esteban
EL KADRI Dany
FADEL Mia
MICELI Florian
SERGENT Julien

TAL – Polytech Paris-Saclay - March 2026

Numbers Speak

$29.3B

Cursor Valuation

$1B+ ARR · Millions of users

$1B

Claude Code Run-Rate

Reached in just 6 months after launch

This is not a niche trend. It's a fundamental shift in how software gets built.
How did we get here and how does this landscape look like?

Plan

1	**Mia**	General Landscape
2	**Esteban**	Tools’ Core Features
3	**Florian**	Cursor vs Windsurf vs Copilot
4	**Luc**	Extensions & CLI Tools
5	**Julien**	Developer Experience
6	**Dany**	Pricing & How to Choose

Why Now? Three Convergences

Better Models

GPT-3 (2020) could barely write a function. By 2024, GPT-4 and Claude 3.5 architect entire features across multiple files.

Lower Costs

Competition between OpenAI, Anthropic, and Google drove prices down.
→ AI coding assistance became viable at $10–20/month for individual developers.

Autocomplete → Agents

Tools evolved from finishing your line of code to reading entire codebases, running tests, finding bugs, and fixing them autonomously. The biggest leap.

From 'smart autocomplete' to 'AI developer' — that shift defines this era.

The Ecosystem: 3 Categories

IDE Forks

Replace your editor

Full VS Code forks rebuilt with AI baked into every interaction

Cursor · Windsurf

Extensions

Add AI to your editor

Plugins that add AI capabilities to your existing VS Code setup

Copilot · Cline

CLI Tools

Work from terminal

Terminal-based agents that execute tasks via natural language

Claude Code · Aider

Tools’ Core Features

The shared capabilities across all AI coding tools

The Feature Stack

Code Completion

AI predicts and completes code as you type — from single lines to entire functions

Inline Editing

Select code and describe changes in natural language. The AI rewrites it in place

Chat with Codebase

Ask questions about your project. The AI reads your files and provides contextual answers

Agent Mode

Autonomous multi-step execution: plan, code, test, fix, commit — across multiple files

Context Management

Codebase indexing, file references, and RAG to give the AI full project understanding

From Autocomplete to Agent

Autocomplete / Chat

- Reactive — waits for you to type
- Works line-by-line or block-by-block
- No project-wide understanding
- You copy, paste, and integrate manually
- *Like a colleague whispering suggestions*

Agent Mode

- Proactive — takes initiative and drives
- Works across multiple files at once
- Reads codebase, understands structure
- Plans → codes → tests → fixes → commits
- *Like a junior dev you assign tasks to*

Context Management

Codebase Indexing

The AI scans and indexes your entire project — folders, files, dependencies — to understand the structure before making suggestions.

File References

Tag specific files (@file) to give the AI focused context. Essential for large projects where you want precise, relevant answers.

RAG & Docs

Retrieval-Augmented Generation pulls in documentation, readmes, and project rules. The AI doesn't just guess — it references your actual docs.

Context is everything — the better the AI understands your project, the better its output.

Head-to-Head Comparison

Cursor vs Windsurf vs GitHub Copilot

Side-by-Side Comparison

	Cursor	Windsurf	GitHub Copilot
Architecture	VS Code fork	VS Code fork	Extension / Plugin
Agent Mode	Composer Agent	Cascade	Coding Agent
Autocomplete	Tab (proprietary)	SWE-1 models	Standard
Multi-model	Yes (many)	Yes	Yes (GPT, Claude, Gemini)
Context Window	128K–200K	Auto-indexed	Varies by model
Standout	Tab predictions	Cascade autonomy	Price and simplicity
Pricing	$20/mo Pro	$15/mo Pro	Free tier + $10/mo

Strengths & Trade-offs

Cursor

✓ Best Tab completions
✓ Most updates and innovations
✓ Large community
✓ Supports “OpenAI-compatible API” (Ollama, LMStudio)

✗ Higher price ($20/mo)
✗ Can feel overwhelming

Windsurf

✓ Cascade agents
✓ Good for big codebases
✓ Cheaper than Cursor ($15/mo)

✗ Uncertain future, small community
✗ UI/UX

GitHub Copilot

✓ Free for Students, most affordable
✓ Simple

✗ Evolves late
✗ No real innovation, a lot of copying cursor

Extensions & CLI

The open-source & terminal approach

Cline: The Open-Source Agent

Why Cline?

5M+ downloads, +250 contributors. Model-agnostic: Claude, GPT, Gemini, or local via Ollama. Switch models mid-session based on task complexity.

Cost Transparency

No opaque credits. See the model used, tokens consumed, and exact dollar cost in real time. A complete break from Cursor and Copilot pricing.

Capabilities

Plan/Act modes for control. File creation & editing. Terminal command execution. Context management. Memory bank. MCP integration for custom workflows and external services.

Trade-offs

Open-source = rougher edges. Can get stuck in loops. Requires prompt crafting skill. Agent decides when 'done' — sometimes too early.

Claude Code: Terminal Powerhouse

Key Strengths

Most powerful CLI agent for complex reasoning. 1M-token context window — ingest an entire codebase in one conversation.

Autonomous multi-step: plans, implements, tests without intervention.

Pay-per-token transparency.

Flags: --max-budget-usd, --worktree, --continue

CLI Alternatives

Aider — Git-intensive workflows. Automatic commits, multi-file editing. Built for refactoring.

OpenCode — 75+ LLM providers, no vendor lock-in.

Plandex — Mega-projects with up to 2M tokens of context.

All open-source, all terminal-first.

Zero friction with Git workflows · Privacy-first (can run 100% local) · No IDE dependency

Not Replacement — Complementarity

Different tools for different moments	
Daily inline suggestions	Cursor / Copilot
Autonomous agent in VS Code	Cline
Deep architectural refactor	Claude Code CLI
Git-heavy multi-file editing	Aider
Privacy-first / air-gapped	Cline + Ollama (local)

In 2026, the winner is the ecosystem. Choose by workflow, not by hype.

Developer Experience

Real-world workflows, tips & pitfalls

Day-to-Day: When to Use What

Use Autocomplete When...

Powered by Fill-In-the-Middle (FIM) architecture. Reads prefix and suffix to predict the gap in real-time.

Rule: Speed > Precision. Accept with Tab, reject with Esc. Never waste time debugging it.

Use Agent Mode When...

Autonomous execution via ReAct (Reasoning and Acting) loops. Has active access to your file system and terminal.

Rule: Best for multi-file refactors. You define the architecture, the agent drives the implementation.

Use Chat When...

Your local search engine enhanced by Retrieval-Augmented Generation (RAG). Indexes your entire codebase.

Rule: Ideal for onboarding, architectural Q&A, and exploring unfamiliar design patterns.

Use CLI When...

Terminal-first agents (like Claude Code or Aider) operating entirely outside the visual IDE.

Rule: Built for heavy-lifting. Perfect for massive codebase-wide migrations and CI/CD workflows.

Real-World Impact: The Productivity Paradox

The Promise (Stats)

Massive Adoption: >50% of professional developers now use AI tools daily *(Stack Overflow 2025/2026)*.

Speed Boost: Up to 40-55% faster on isolated coding tasks.

Reduced Fatigue: Automating boilerplate logic drastically lowers cognitive load.

The Reality (The Paradox)

The PR Bottleneck: AI generates code instantly, creating massive traffic jams during Code Reviews (Pull Requests).

Technical Debt: Without strict CI/CD and testing, AI accelerates the delivery of "spaghetti code" to production.

The Role Shift: Developers transition from *Code Writers* to *Code Reviewers & System Architects*.

Pro Tips & Critical Pitfalls

Best Practices (Pro Hacks)

Industrialize Context: Use `.cursorrules` or `.clinerules` in your root folder. Force the AI to respect your specific architecture and team conventions.

Manage Context Windows: Long threads degrade AI attention (Needle in a Haystack problem). **Rule:** 1 Feature = 1 New Chat Thread.

Trust, but Verify: Treat all AI outputs as code written by an enthusiastic but junior developer.

Watch Out For

Hallucinations: AI hates empty spaces. It will invent APIs or packages that look syntactically perfect but don't exist.

Security Leaks: Unrestricted IDEs will send your secrets to the cloud. Always configure `.cursorignore` for `.env` files and API keys.

The 'Accept All' Trap: Blindly pressing Tab without reading the generated logic.

Pricing & Selection

How to choose the right tool for your needs

Pricing at a Glance

Tool	Free Tier	Pro / Paid	Model
Cursor	Limited trial	$20/mo	Subscription
Windsurf	5 req/day	$15/mo	Credits
GitHub Copilot	2000 completions	$10/mo	Subscription
Cline	Free (open-source)	API costs only	Pay-per-token
Claude Code	None	$20/mo (Pro plan)	Pay-per-token
Aider	Free (open-source)	API costs only	Pay-per-token

How to Choose

Solo dev, budget-conscious	→ **Cline + own API keys**
Team / Enterprise	→ **Cursor or Copilot Enterprise**
Deep in GitHub ecosystem	→ **GitHub Copilot**
Terminal-native workflow	→ **Claude Code**

Key Takeaways

1. AI coding tools are no longer optional — they're reshaping the industry
2. Three categories: IDE forks, extensions, and CLI tools — each for different needs
3. The shift from autocomplete to agents is the defining change of this era
4. Start now: try free tiers, experiment, and find what fits your style

Questions?